

INFORME TÉCNICO - PROYECTO INTEGRAL S.I.G.I.

Municipalidad Valle del Sol - Gestión y prevención de emergencias

Campo | Dato

Carrera |

Índice

1. Contexto

2. Objetivos

1. Contexto

La Municipalidad Valle del Sol es un caso semestral ficticio, pero el problema que plantea es

real: en m

La Subdirección de Gestión de Emergencias y Prevención de Desastres necesita una plataforma

donde:

- Los residentes puedan registrarse, reportar emergencias con detalle y foto, ver el estado de

sus repor

Nosotros tres (Hawk, Emilio y Rodrigo) desarrollamos S.I.G.I. (Sistema Inteligente de Gestión

de Incen

Este informe documenta el proyecto completo (no solo backend): qué hace cada parte, por qué lo

dejamos

2. Objetivos del proyecto

2.1 Objetivo general

Armandos una aplicación web full stack que concentra el ciclo de un reporte ciudadano: aviso

inicial, va

2.2 Objetivos específicos

Objetivo | Cómo lo abordamos

Autentica
EQUIPO

3. Arquitectura general

El cliente (navegador con React) solo habla con el puerto 8080 del API Gateway. Eureka

registra l

flowchart LR

subgrap

GW --> EU

GW -->

Principio database per service: cada microservicio tiene su base (db_usuario, db_reporte,

etc.). En

Servicio | Puerto | Base de datos | Responsabilidad principal

eureka |

4. Backend - microservicios y funciones

En esta sección explicamos las clases y métodos que nos importaron para entender el sistema.

No listam

4.1 API Gateway - `JwtAuthGatewayFilterFactory`

Elemento | Qué hace | Por qué existe

apply(Co
Cloud Ga

Si el token falta o es inválido, respondemos 401 y no llegamos al microservicio.

4.2 servicio-usuario

AuthController

- registrar(UsuarioRegistroDTO) - Crea cuenta. Validamos con @Valid y devolvemos 201 o 400 si

el email y

UsuarioService

- registrar - Hashea la contraseña con BCrypt antes de guardar. Nunca persistimos texto plano.

- login - C

UsuariosPruebaInitializer (CommandLineRunner)

- Al arrancar, crea si no existen los usuarios de Hawk, Emilio y Rodrigo con contraseñas

conocida

4.3 servicio-reporte

ReporteService.crearReporte

1. Llama a UbicacionConsultaService (Feign + circuit breaker) para obtener lat/lon.

2. Arma l

listarMisReportes - Un ciudadano solo ve los suyos; operador y admin pueden consultar por

usuariolo

listarPendientes - Cola para la central (Emilio en las pruebas).

listarTodos - Solo admin/operador; alimenta el dashboard de Rodrigo.

validar - Si aprobado == true, pasa a VALIDADO y llama a EmergenciaClient.crearDesdeReporte

para abri

4.4 servicio-ubicacion

UbicacionService.obtenerCoordenadas

- Consulta OpenCage con la dirección del reporte.

- Guarda

4.5 servicio-emergencia

EmergenciaService.crearDesdeReporteValidado - Instancia una emergencia ligada al reportId.

actualizarEstado - Lo usa el equipo de emergencia desde el frontend (/emergencias) para pasar

de ACTIV

4.6 servicio-recurso

RecursoService.asignarAutomaticamente - Según prioridad del reporte, busca un recurso

disponibl

4.7 servicio-notificacion

NotificacionService.crearAlertaPorEmergencia - Cuando se valida un reporte, registra

notificaci

4.8 servicio-empleo

EmpleoService

Método | Función

listarActiv

DatosEjemploEmpleos - Carga tres avisos iniciales al primer arranque (brigadista, operador

SIGI, adr

4.9 servicio-media

MediaService.subir

- Recibe MultipartFile, genera nombre UUID en disco (/app/uploads en Docker con volumen

persisten

obtenerArchivo - Sirve el binario para que el frontend muestre la imagen en <img

src="/api

Por qué un servicio aparte: las fotos no deberían ir en Base64 dentro del JSON del reporte;

ensucia l

5. Frontend - componentes, servicios y hooks

Stack: React 19, Vite, Tailwind CSS, React Router. Conectamos al backend vía proxy de Vite

(/api y /a

5.1 Capa de servicios (`src/services/`)

Archivo / función | Qué hace | Por qué

apiClient
token

5.2 Contexto de autenticación - `AuthContext.jsx`

Función / estado | Rol

auth | Ob

Usamos Context para no pasar el token en props por diez niveles. En un proyecto más grande

evaluaría

5.3 Patrón del curso: loading, error, datos

En MisReportes, Dashboard, Empleos y Emergencias repetimos el mismo esquema que vimos en Full

Stack III

```
const [data, setData] = useState([]);
```

const [loa

```
useEffect(() => { cargarDesdeApi(); }, [dependencia]);
```

Por qué: el usuario siempre debe ver si está cargando, si falló la red o si la lista llegó

vacía. No

5.4 Componentes principales

Componente | Responsabilidad

ReporteI
local

5.5 Páginas por rol

Ruta | Usuario típico | Función

/login, /re

5.6 Utilidades - `reporteMappers.js`

reporteApiACard(reporte) - Convierte la respuesta del backend (prioridad: CRITICA, direccion)

al format

5.7 Constantes - `usuariosPrueba.js`

Centraliza emails y roles de Hawk, Emilio y Rodrigo. La pantalla de login tiene botones que

rellenan

6. Seguridad y roles de usuario

Rol | Permisos en la práctica

CIUDAD.

La autorización fina en microservicios es básica (revisamos X-User-Role en algunos endpoints).

El Gatew

Usuarios de prueba (semilla al arrancar backend):

Persona | Email | Rol

Hawk Du

7. Flujos operativos principales

7.1 Reporte con foto (ciudadano)

1. Login -> token en localStorage.

2. En Nu

7.2 Validación y emergencia (operador)

1. Emilio entra al Dashboard o Cola reportes.

2. GET /a

7.3 Postulación a empleo

1. Hawk entra a Empleos -> GET /api/empleos.

2. POST

8. Pruebas realizadas

8.1 Backend

- JUnit 5 + Mockito en servicios críticos (ReporteService, UsuarioService,

Notificaci

8.2 Frontend

- Jest + Testing Library: login, registro, ReporteIncendioCard, mappers, servicios empleo y

media co

Las pruebas E2E con navegador automatizado no las alcanzamos a implementar; las dejamos como

mejora.

9. Despliegue y ejecución

Backend (desde sigi-backend/)

docker co

Frontend (desde Sigi_Front/)

npm insta

- Gateway: http://localhost:8080

- Fronten

Variables relevantes: JWT_SECRET (igual en gateway y usuario), OPENCAGE_API_KEY (opcional,

mejora m

10. Limitaciones y trabajo futuro

1. Actividades municipales siguen en localStorage del frontend; no tienen microservicio propio

todavía.

11. Conclusiones

Logramos entregar una plataforma coherente con el enunciado del caso semestral: residentes

reportan

En frontend aplicamos de forma real lo visto en Full Stack III: componentes reutilizables,

estado lo

Como equipo nos quedó claro que el valor del sistema no está solo en "tener muchos servicios",

sino en tr

12. División del trabajo en el equipo

Distribución aproximada según lo que cada uno fue llevando en el repo (no es rígido; nos

reunimos

Integrante | Aportes principales

Hawk D
pruebas

La integración final (conexión frontend-backend, servicio-empleo, servicio-media) la hicimos

Informe elaborado por Hawk Durant, Emilio Jaramillo y Rodrigo Candia - Proyecto S.I.G.I.,

Municipa